Data Flow Diagram (DFD) – DarshanEase: Smart Temple Darshan Ticket Booking System

Introduction

A Data Flow Diagram (DFD) is a structured, graphical representation used in software engineering to model how information moves through a system. It emphasizes the **flow of data** between external actors (users or external systems), internal processes (modules/services), and persistent storage (databases). In system analysis and design, DFDs help stakeholders understand **what data is required**, **where it originates**, **how it is transformed**, and **where it is stored**, without being tied to specific implementation details.

For **DarshanEase**, a Full Stack **MERN** (MongoDB, Express.js, React, Node.js) web application for temple darshan ticket booking, QR-code ticketing, donations, and admin/organizer management, DFDs visualize how requests such as registration, slot selection, booking confirmation, QR generation, and administrative reporting move through the application modules and are persisted in **MongoDB**. This report documents the DarshanEase DFDs at multiple levels (Context/Level 0, Level 1, and Level 2) to support implementation, testing, and maintenance.

SECTION 1 – Overview of Data Flow Diagrams

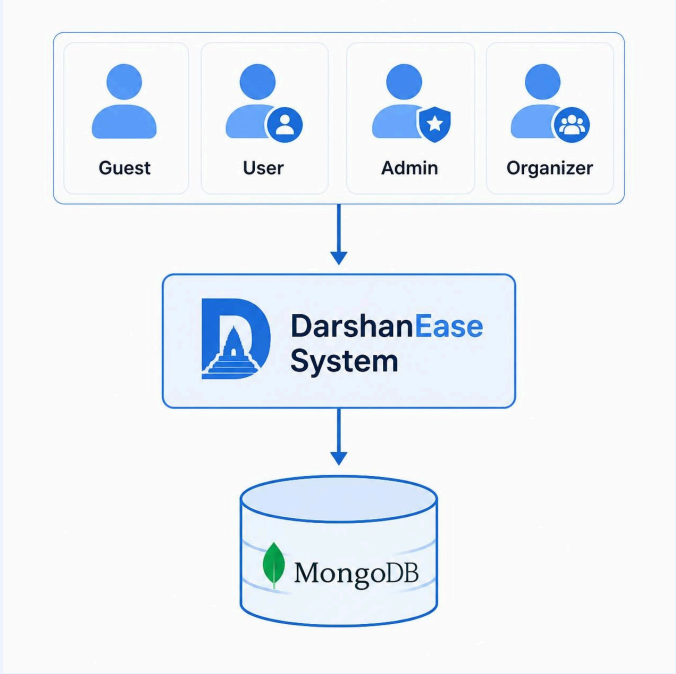
1. 1.1 **Definition of a DFD** — A DFD is a logical model that shows how data enters a system, how it is processed, and how it is stored and output. In DarshanEase, examples include the flow from a user's booking request to booking creation and QR-code ticket generation.
2. 1.2 **Objectives of a DFD** — (i) clarify system boundaries, (ii) identify processes and data stores, (iii) document data exchanges, and (iv) ensure that functional requirements (e.g., booking, donations, verification) are consistently handled across modules in DarshanEase.
3. 1.3 **Core Components**
   - ▫️ **External Entities** — actors outside the system boundary (e.g., Guest User, Registered User, Organizer, Administrator).
   - ● **Processes** — transformations of data (e.g., Authenticate User, Manage Slots, Confirm Booking, Generate QR/PDF).
   - ⬚ **Data Stores** — persistent repositories (MongoDB collections such as Users, Temples, Slots, Bookings, Donations).
   - → **Data Flows** — directed movement of information between entities, processes, and stores (e.g., "Booking Request", "Payment Status", "Ticket PDF").

4. 1.4 **Usefulness in System Analysis & Design** — DFDs support requirement validation, modular decomposition, and traceability of data handling. For DarshanEase, they ensure that sensitive operations (authentication, payment/donation records, QR ticket validation) follow controlled paths and are reliably persisted and auditable in MongoDB.

SECTION 2 – Context Diagram (Level 0 DFD)

The **Context Diagram (Level 0 DFD)** provides the highest-level overview of DarshanEase by representing the entire application as a single process ("DarshanEase System") and showing how external entities exchange data with it. At this level, internal modules are not decomposed; instead, the emphasis is on system boundary, major actors, and primary information exchanges. All incoming requests are processed by the DarshanEase system and relevant records are stored within **MongoDB** for consistency, traceability, and reporting.

1. 2.1 **External Entities and Interactions**
    - ⬜ **Guest User** — initiates **Registration** (user details), sends **Login Request** (credentials), and performs **Temple Search** (query parameters). The system returns search results, authentication outcomes, and public temple information.
    - ⬜ **Registered User** — sends **Booking Request** (temple, slot, devotee details), performs **Donation** (amount, payment reference), submits **Profile Update** (contact details), and requests **Ticket Download** (booking ID). The system returns booking confirmation, QR/PDF ticket, donation receipt, and updated profile data.
    - ⬜ **Organizer** — performs **Slot Management** (create/update slot capacity and timings), executes **QR Ticket Verification** (scan/validate QR payload), and conducts **Booking Monitoring** (slot-wise booking counts and attendance). The system returns verification status and operational dashboards.
    - ⬜ **Administrator** — handles **User Management**, **Temple Management**, **Booking Management**, generates **Reports**, and performs **Donation Monitoring**. The system returns analytics summaries and management actions status, backed by MongoDB records

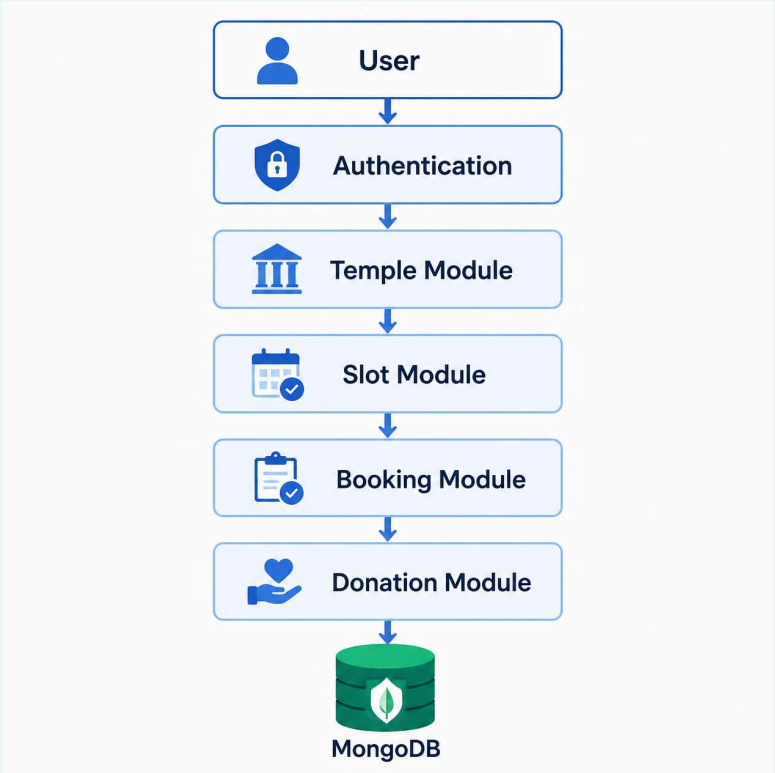


SECTION 3 – Level 1 Data Flow Diagram

The **Level 1 DFD** decomposes the DarshanEase system into major internal modules and shows how data moves between them and MongoDB. This level is aligned with the MERN architecture where the React client interacts with a Node/Express backend through RESTful APIs, and data is persisted in MongoDB collections.

1. 3.1 **End-to-End Request Flow (Implementation-Aligned)**
   - → User submits request via **React Frontend** (UI forms and actions).
   - → Request is sent to **REST API** endpoints (Express routes).
   - → **Express Controller** validates inputs and forwards to business services.
   - → **Business Logic** enforces rules (slot capacity, payment status, role checks).
   - → Data is read/written in **MongoDB** (Mongoose models/collections).
   - → Response (status, data, ticket links) is returned to frontend for rendering and download.
2. 3.2 **Module-Level Explanation (DarshanEase)**
   - ● **Authentication Module** — registration/login, JWT/session handling, role-based access (User/Organizer/Admin), password hashing, and secure profile updates.
   - ● **Temple Management Module** — CRUD for temples, metadata, rules, and public browsing/search indexes for devotees.
   - ● **Darshan Slot Module** — slot creation, capacity settings, date/time windows, and availability calculations used before booking confirmation.
   - ● **Booking Module** — booking lifecycle: validation, seat allocation, booking ID generation, ticket status tracking, and booking history retrieval.
   - ● **Donation Module** — donation initiation, payment record persistence, receipt generation, and reporting for admins/temple accounts.

- ● **Notification Module** — confirmation messages and delivery records (email/SMS/WhatsApp integrations can be logged for auditability).
- ● **Admin Dashboard** — governance workflows: manage users/temples/bookings, view donation summaries, generate reports and analytics.
- ● **Organizer Dashboard** — operational workflows: manage slots, monitor bookings, and verify QR tickets at entry checkpoints.
- ⬚ **Database (MongoDB)** — persistent storage of users, temples, slots, bookings, donations, notifications, and feedback for traceability and reporting.

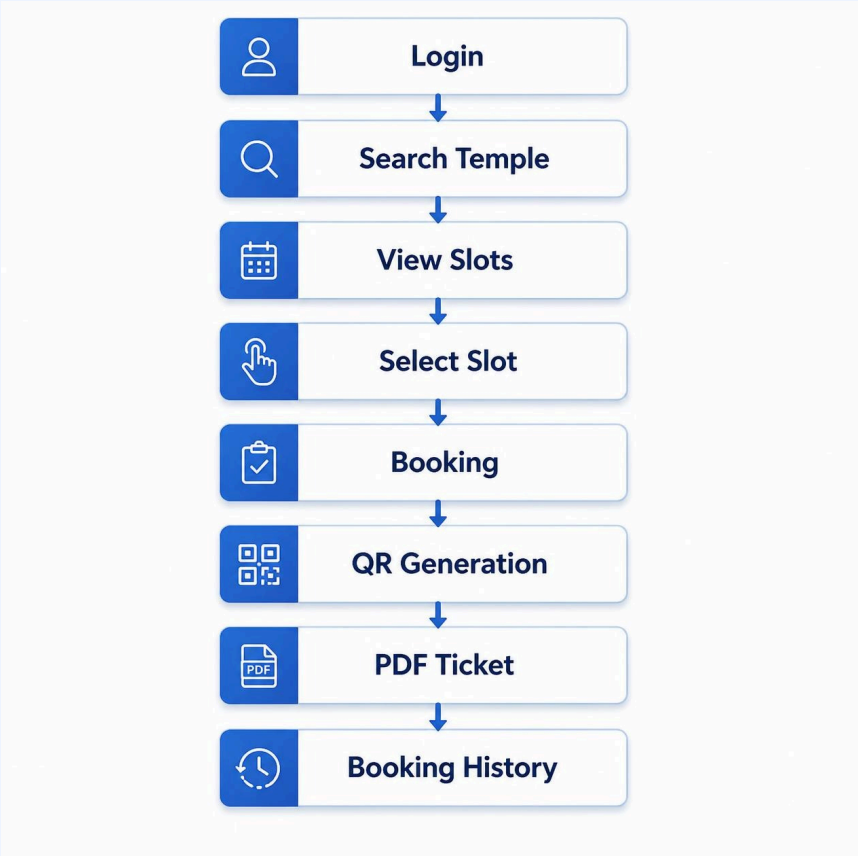


SECTION 4 – Level 2 Data Flow Diagram

The **Level 2 DFD** further decomposes a critical process into detailed steps. For DarshanEase, the **Booking Process** is central because it must accurately manage slot capacity, generate verifiable QR-code tickets, and maintain an auditable booking history in MongoDB. The steps below describe the expected data transformations and storage operations.

1. 4.1 **Detailed Booking Sequence (DarshanEase)**
   a. 1) **User Login** — user submits credentials; authentication verifies identity and role; session/JWT is issued.
   b. 2) **Search Temple** — user provides search filters; system returns matching temples and basic availability cues.
   c. 3) **View Temple Details** — system provides temple profile, rules, schedule, and available darshan slots.

d. 4) **Select Darshan Slot** — user selects date/time slot and ticket quantity (as permitted).
e. 5) **Check Seat Availability** — Booking Module queries Slots data to ensure capacity is sufficient and slot is active.
f. 6) **Enter Devotee Details** — devotee names/IDs (as applicable) are captured for the ticket record and verification.
g. 7) **Confirm Booking** — system validates inputs, computes totals/fees (if any), and locks the booking intent.
h. 8) **Store Booking** — booking record is created in MongoDB with user, temple, slot, devotee list, and status.
i. 9) **Reduce Slot Capacity** — slot capacity is decremented atomically to prevent overbooking under concurrency.
j. 10) **Generate Booking ID** — a unique identifier is issued for retrieval, audit, and ticket association.
k. 11) **Generate QR Code** — QR payload encodes booking ID (and verification hash/claims) for organizer validation.
l. 12) **Generate PDF Ticket** — a downloadable ticket is produced with temple/slot details and embedded QR code.
m. 13) **Update Booking History** — user profile references booking; activity logs/notifications are recorded for traceability.
n. 14) **Display Confirmation** — frontend renders confirmation screen and provides ticket download link and status updates.



SECTION 5 – Data Stores

DarshanEase uses **MongoDB** as its primary data store. Each functional area is represented as a collection that supports application workflows (booking, verification, donations) and administrative reporting. The following table summarizes the principal collections used by the system.

Data Store	Purpose	Stored Information
Users	Identity & access management	Profile data, hashed credentials, roles (user/organizer/admin), booking references
Temples	Temple catalog & browsing	Temple details, location, rules, organizer mapping, visibility status
Darshan Slots	Schedule and capacity control	Date/time windows, capacity, remaining seats, status, pricing/constraints
Bookings	Ticket issuance & audit trail	Booking ID, user/temple/slot linkage, devotee details, status, QR payload, ticket URL
Donations	Donation tracking and receipts	Donor user (optional), temple, amount, payment reference, timestamp, receipt metadata
Notifications	Delivery and confirmation logs	Booking/donation confirmations, delivery channel, delivery status, timestamps
Feedback	Quality and support improvement	Ratings, comments, issue categories, user references, admin resolution status

SECTION 6 – Data Flow Explanation

The table below lists key logical data flows in DarshanEase, showing the direction of movement, the data transferred, and its purpose with concrete examples tied to the MERN-based implementation.

Source	Destination	Data Transferred	Purpose	Examples
Guest	Authentication	Registration data, login request	Create account / initiate session	Name, phone/email, password; login credentials
Authentication	Users Collection	User record updates	Persist identity and roles	Hashed password, role=Registered User, JWT refresh metadata
User	Booking Module	Booking request payload	Start booking transaction	Temple ID, slot ID, devotee list, quantity
Booking Module	Bookings Collection	Booking record	Persist booking and status	Booking ID, status=Confirmed, ticket URL, timestamps
Booking Module	QR Generator	QR payload	Enable entry verification	Encoded booking ID + verification hash/claims
QR Generator	User	QR code + PDF ticket	Deliver ticket for download/use	PDF with temple/slot details and embedded QR
Organizer	Slot Module	Slot updates, capacity rules	Maintain operational schedule	Create slots, adjust remaining seats, close slot
Admin	Temple Module	Temple CRUD commands	Govern temple catalog	Add temple, update rules, assign organizer
Admin	Reports	Report	Generate	Bookings per

		parameters	analytics output	temple/date, donation totals, verification counts
Donation Module	Donations Collection	Donation record	Persist donation for receipt & audit	Amount, temple ID, payment reference, timestamp

SECTION 7 – Advantages of the DFD Design

The DFD design enhances understanding of DarshanEase by presenting an unambiguous view of how data is captured, validated, transformed, and stored across user-facing features and administrative workflows. By explicitly mapping external entities (Guest, Registered User, Organizer, Administrator) to system processes, the DFD supports accurate requirement verification and ensures that each role only accesses the data and functions aligned with its permissions.

From a development perspective, the DFD enables **modular implementation** by clarifying boundaries between the Authentication, Booking, Slot, Donation, and Dashboard modules. This promotes parallel development within the MERN stack (React UI, Express controllers, service-layer business logic, and MongoDB models) and reduces integration errors. The diagrams also simplify **debugging and testing** because each data flow can be traced end-to-end—e.g., booking confirmation can be validated from the API request payload through MongoDB writes and finally to QR/PDF generation and user delivery.

Furthermore, the DFD provides high-quality project documentation that improves maintainability. Future enhancements—such as advanced payment gateways for donations, multi-temple organizer mapping, or enhanced verification analytics—can be incorporated with minimal ambiguity because the existing DFD serves as a stable reference for current data pathways and persistence requirements.

SECTION 8 – Expected Outcome

The Data Flow Diagrams documented in this report clearly describe the interaction between DarshanEase users, application modules, and the MongoDB data stores. The DFDs demonstrate how DarshanEase securely handles **user authentication**, **temple browsing**, **darshan slot management**, **ticket booking**, **QR-code ticket generation**, **donation management**, and **administrative/organizer operations** through controlled, traceable flows.

As a result, the DFD serves as an essential blueprint for implementing and maintaining the DarshanEase application within a MERN architecture. It supports consistent development

practices, strengthens auditability for bookings and donations, and provides a reliable foundation for future scalability and feature expansion.